```
  function RandomizedTreap()
    this.treap ← new Treap
```

Take a look at figure 3.13 to check out a more balanced version of the tree produced at the end of figure 3.9, by replacing those priorities with randomly generated real numbers. It's also possible to use random integers for priorities, but using real numbers reduces the possibility of ties, and improves the final result.

We will see in section 3.5 that, if the priorities are drawn from a uniform distribution, the expected height of the tree is logarithmic in the number of nodes.

The best part is that we have already written almost all the code the need to implement this new data structure: we can internally use a treap (listing 3.9) and all its API methods will be just wrappers for treap's methods, except for `insert`, that's the only one for which we need to write an extra line of code.

As it's shown in listing 3.10, all we need to do is generate a random priority when we insert a new key in our randomized treap.

You can find a Java implementation for RTs on the book's repo on GitHub.

Listing 3.10 method RandomizedTreap::insert

```
function insert(key)
                  #1
  return this.treap.insert(key, this.randomGenerator.next())
         #2
```

#1 Method `insert` takes the key to insert into the tree; it doesn't return anything, but has side effects on the tree.